

Betriebssysteme

Prozesse, Threads und Scheduling

Andreas Scheibenpflug, MSc
SE Bachelor (BB/VZ), 2. Semester

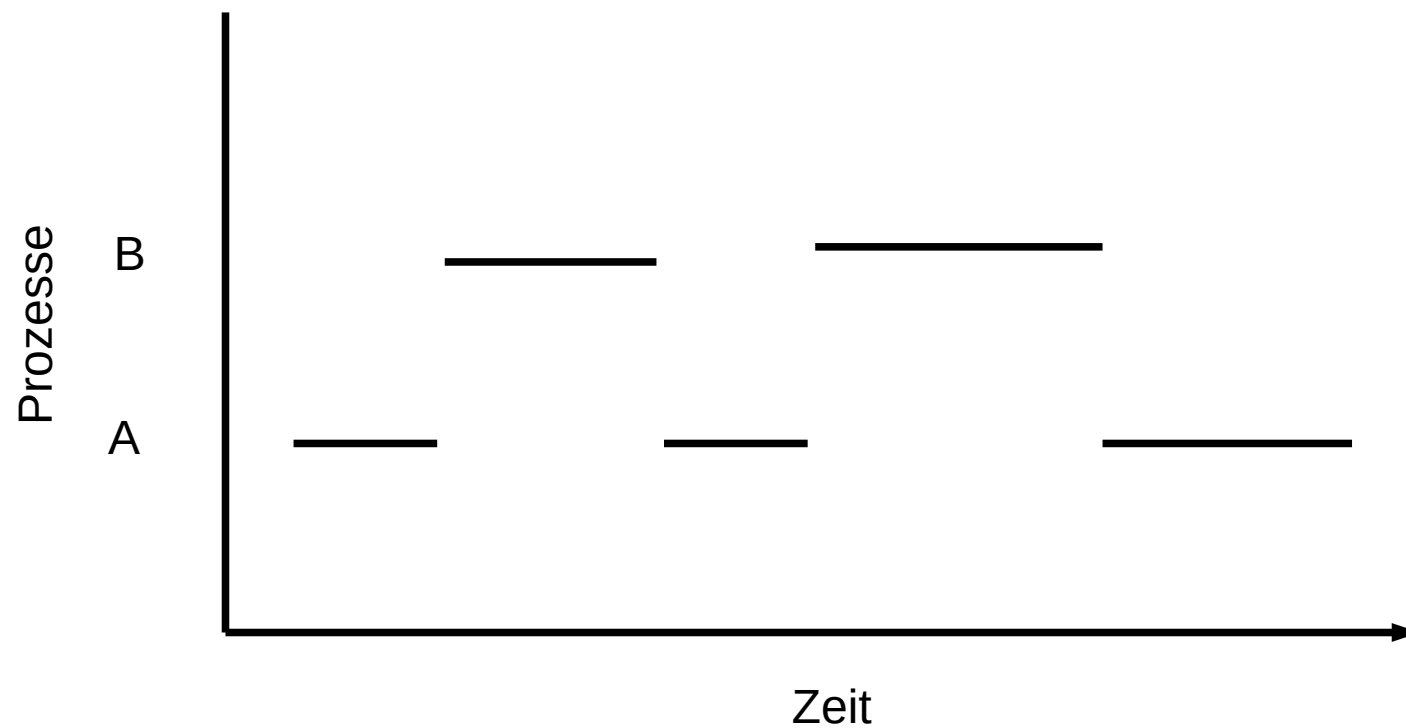
Prozess

- Abstraktion eines laufenden Programms
- Erlauben Pseudoparallelität wenn z.B. nur ein Prozessor vorhanden ist
- Gewöhnlich laufen auf einem Computer mehr Prozesse als Prozessoren/Kerne verfügbar sind
- OS kümmert sich darum, dass jeder Prozess CPU Zeit bekommt (Scheduling)
- Für den/die BenutzerIn sieht es so aus, als ob alle Prozesse gleichzeitig laufen

Prozessmodell

- Ein Prozess ist eine Instanz eines Programms zur Laufzeit und besteht aus
 - Code/Befehlszähler (Instruction Pointer)
 - Instruktion die als nächstes ausgeführt wird
 - Register
 - Variablen (Daten)
- Dem OS steht ein Prozessor zur Verfügung
- OS führt eine Menge an Prozessen pseudo-parallel aus
 - Jeder Prozess hat die CPU für eine begrenzte Zeit zur Verfügung
 - OS teilt Prozessen die CPU zu und wechselt die Prozesse aus
 - Zu einem bestimmten Zeitpunkt wird immer nur ein Prozess ausgeführt

Prozessmodell



Prozesserzeugung

- Vier Ereignisse die zum Erzeugen eines neuen Prozesses führen
 - Initialisierung des OS
 - Beim Starten des OS werden Prozesse erzeugt
 - System Calls
 - Prozess, der einen anderen Prozess erzeugt (z.B. Linux: `fork()`, Windows: `CreateProcess()`)
 - Durch den/die BenutzerIn
 - z.B. Starten eines Programms in der Shell
 - Start eines Stapeljobs
 - z.B. Batchprocessing bei Großrechnern (Mainframes)
- Technisch gesehen meist Variante 2 (System Calls)

Prozesserzeugung - Beispiel

- `fork` dupliziert den Prozess inkl. Speicher
- Rückgabewert von `fork` ist
 - 0 im Kindprozess
 - PID des Kindes im Elternprozess
- `execve` erlaubt das Ausführen eines Programms
 - Ersetzt den aktuellen Prozess inkl. Speicher durch das auszuführende Programm

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

// gcc fork.c -o fork
// ./fork
int main(void)
{
    pid_t pid_new;
    int status;
    char *args[] = {"uname", "-a", NULL};
    char *env[] = {NULL};

    pid_new = fork();

    if (pid_new == 0)
    {
        printf("Child process says Hello\n");
        execve("/usr/bin/uname", args, env);
        printf("Never executed code if execve is successful\n");
    }
    else
    {
        printf("Original process says Hello!\n");
        wait(&status);
        printf("Original process: Child finished, exiting...\n");
    }
    return 0;
}
```

```
Original process says Hello!
Child process says Hello
Linux ThinkPad-L390 5.4.0-31-generic #35-Ubuntu SMP Thu May 7 20:20:34 UTC 2020
x86_64 x86_64 x86_64 GNU/Linux
Original process: Child finished, exiting...
```

Prozessbeendigung

wenn der Mutterprozess immer wartet dass das childprozess fertig ist, dann wird er ja nie beendet?
wenn child PID immer 0, wie unterscheiden?

- Vier Ereignisse, die zum Beenden eines Prozesses führen
 - Normales Beenden (freiwillig)
 - Aufgabe beendet (z.B. Linux: `exit(0)`, Windows: `ExitProcess()`)
 - Beenden aufgrund eines Fehlers (freiwillig)
 - Programm stellt Fehler fest und beendet mit z.B. `exit(-1)`
 - Beenden aufgrund eines schwerwiegenden Fehlers (unfreiwillig)
 - z.B. Programmierfehler, Division durch Null, Zugriff auf ungültige Speicheradressen,...
 - Beenden durch einen anderen Prozess (unfreiwillig)
 - z.B. Linux mit `kill()` oder Windows mit `TerminateProcess()`

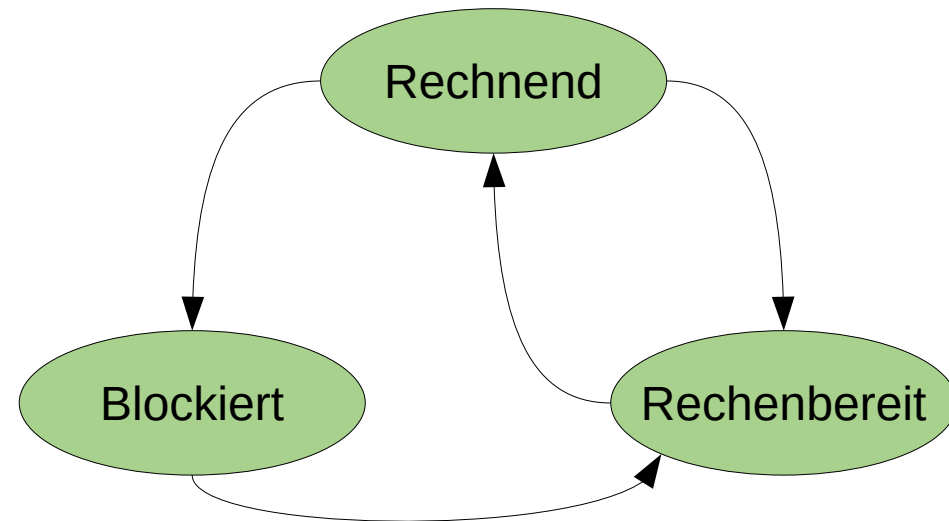
Prozesszustände

- Prozesse müssen oft warten auf
 - Ein- und Ausgabe auf Geräten
 - z.B. Lesen vom Dateisystem, `HTTP GET`, ...
 - Andere Prozesse
 - z.B. `cat file.txt | grep Linux | sort`
- Dieser Zustand wird als „blockiert“ bezeichnet
- Im Gegensatz zum Stoppen eines rechenbereiten Prozesses weil das OS dies entscheidet
 - z.B. Umschalten zu einem anderen Prozess weil aktueller Prozess seinen Anteil an Rechenzeit verbraucht hat

Prozesszustände

▪ Drei Zustände

- **Rechnend**
 - Prozess läuft auf der CPU und führt Befehle aus
- **Rechenbereit**
 - Prozess ist bereit aber gestoppt, weil gerade ein anderer Prozess rechnet
- **Blockiert**
 - Nicht lauffähig, da auf ein Ereignis (z.B. Benutzereingabe, Datei eingelesen,...) gewartet wird



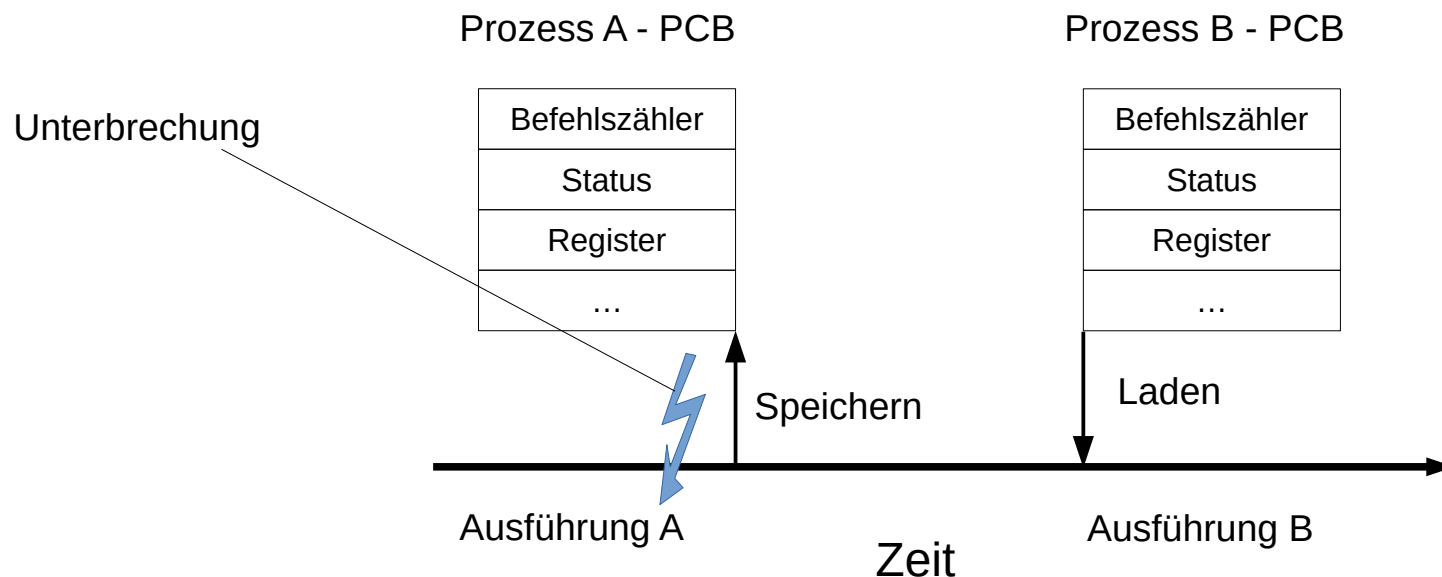
Prozesstabelle

- OS besitzt eine Prozesstabelle, welche eine Liste aller Prozesse enthält
- Ein Eintrag in dieser Liste wird als Prozesskontrollblock (PCB) bezeichnet
 - Enthält alle Informationen (Zustand) zu einem Prozess
 - Ermöglicht damit das Fortsetzen von Prozessen
 - Ermöglicht Scheduling Entscheidungen zu treffen

Prozessverwaltung	Speicherverwaltung	Dateiverwaltung
Register	Zeiger auf Textsegment	Arbeitsverzeichnis
Befehlszähler (= instruction pointer)	Zeiger auf Datensegment	Dateideskriptor
Stackpointer	Zeiger auf Stacksegment	User Id
Programmstatuswort	...	Gruppen Id
Prozess ID (PID)		...
Benutzte CPU Zeit		
Prozesszustand		

Prozesswechsel

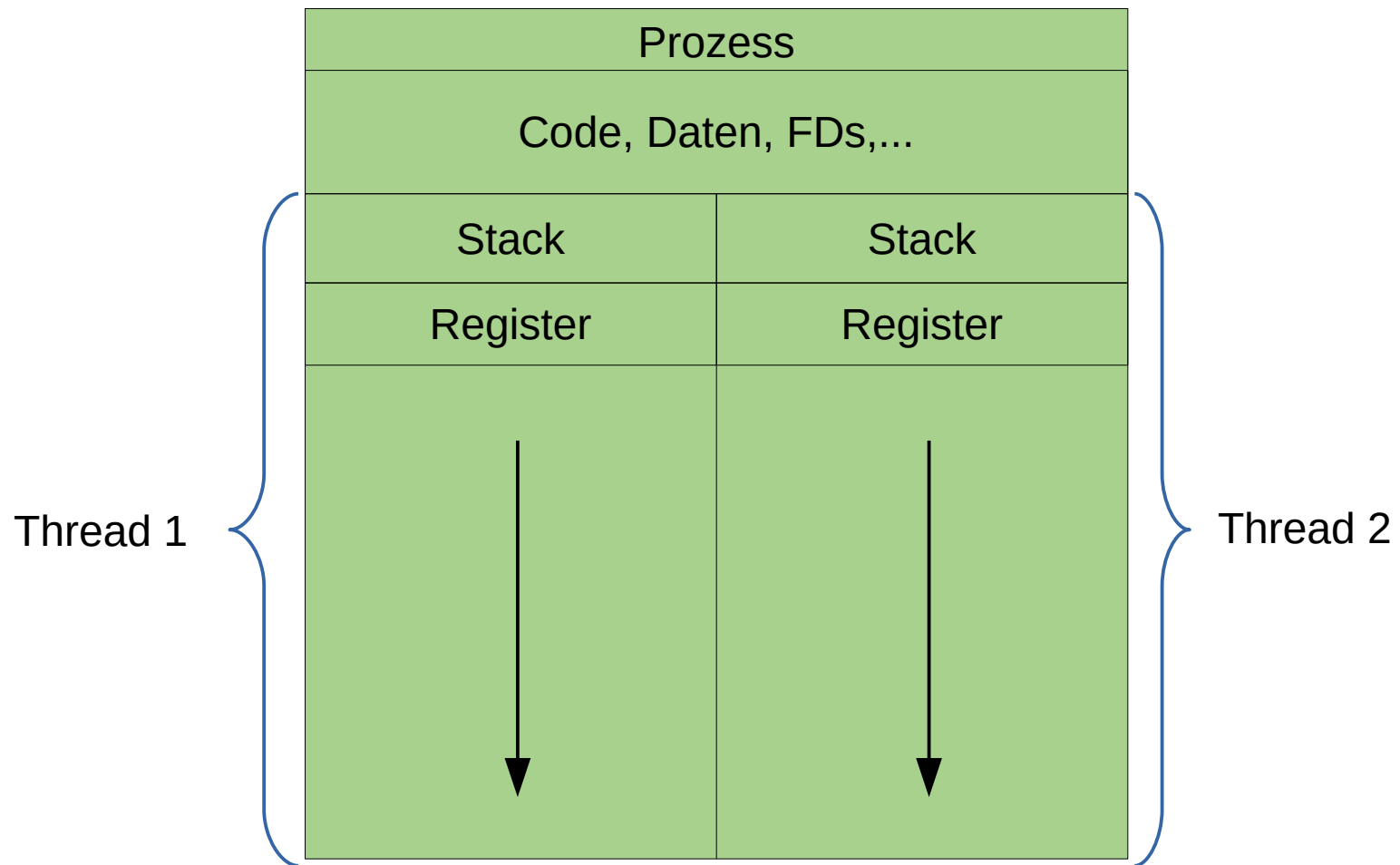
- Anzahl Prozesse > Anzahl CPUs (in unserem Modell == 1)
 - OS muss zwischen Prozessen wechseln, da jeder Prozess CPU Zeit zur Ausführung benötigt
- Prozesswechsel - Context Switch
 - Laufender Prozess wird unterbrochen
 - Scheduler wählt einen anderen, rechenbereiten Prozess aus und führt diesen aus
- Informationen zu unterbrochenem Prozess müssen gespeichert werden
 - Damit dieser später wieder ausgeführt werden kann
 - Zustand wird im PCB gespeichert



Threads

- Jeder Prozess hat seinen eigenen Adressraum („Speicher“)
 - CPU führt eine Instruktion nach der anderen sequentiell aus
- Ein Prozess kann mehrere Threads starten
 - Threads sind parallel laufende Ausführungspfade
 - Im selben Adressraum wie der Prozess
- Vorteile
 - Einfacheres Programmiermodell (im Vergleich zu z.B. IPC)
 - Leichtgewichtiger als Prozesse
 - Performance, z.B. 1 Thread rechnet während ein 2. auf I/O wartet
 - Performance bei Multicore Prozessoren

Threads



Threads

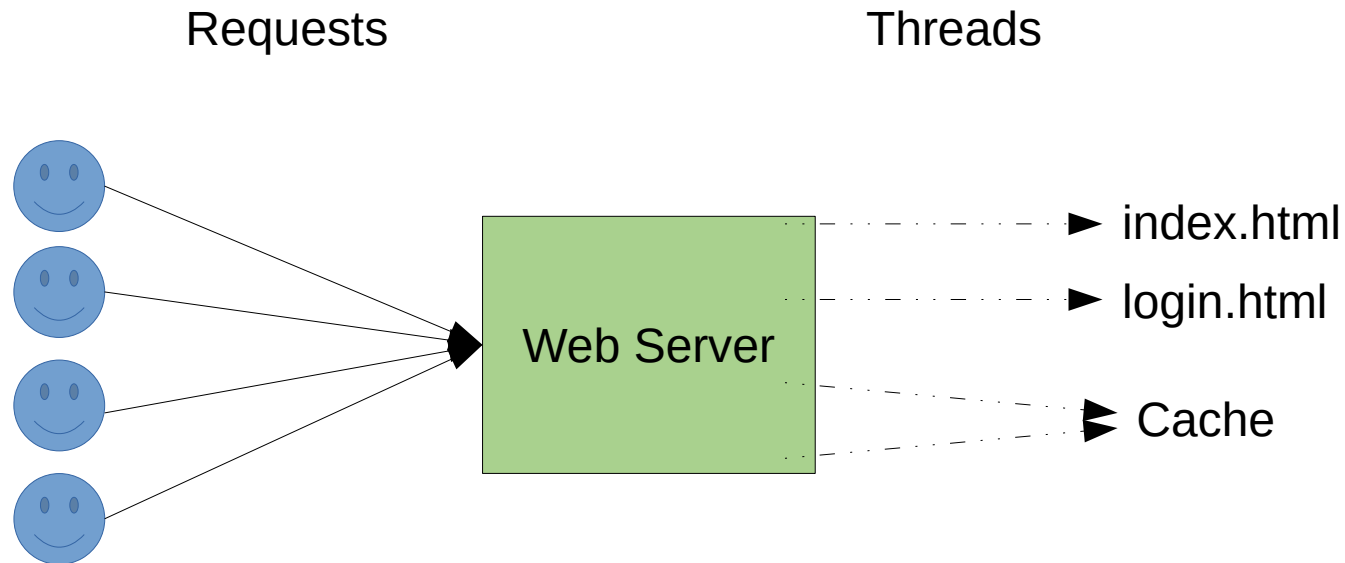
▪ Implementierung

- Im OS: OS hält Thread Tabelle ähnlich zur Prozesstabelle mit z.B.:
 - Stack
 - Register
 - Befehlszähler
 - Threadzustand
- Im User Space: Implementiert Funktionalität in Laufzeitsystem (Bibliothek)
 - Bessere Performance
 - Probleme bei Blocking I/O, Page Faults,... → andere Threads sind auch blockiert
- Hybrid: Kombination aus OS und User Space Threads

Threads - Implementierungen

- Windows
 - „Echte“ OS Threads, Verwenden der `CreateThread(...)` Funktion
 - Jeder Prozess besteht aus mindestens einem Thread
 - Hybrid Threads mit User-Mode Scheduling
- Linux
 - Threads werden durch das OS verwaltet
 - Es wird aber die Implementierung der Prozessverwaltung verwendet
 - Das bedeutet beinahe kein Unterschied zwischen Prozessen und Threads aus Kernel Sicht
 - Verwendung der Funktion `clone(...)` und setzen der Flags für das Erlauben des Zugriffs auf die File Deskriptoren etc. im Prozessadressraum
 - PID bleibt allerdings gleich

Threads - Beispiel



Threads - Beispiel

- Diskussion:
Implementierungsmöglichkeiten
 - 1 Prozess, sequenzielle Abarbeitung
 - Mehrere Prozesse, Parallelität
 - 1 Prozess, mehrere Threads (pro Request?)
 - 1 Prozess, Non-Blocking IO
 - ...

POSIX Threads - Beispiel

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

// gcc -pthread threads.c -o threads
// ./threads

void *say_hello_thread(void* id)
{
    printf("Hi! I'm thread %d\n", id);
    pthread_exit(NULL);
}

int main(void)
{
    static int nr_of_threads = 20;
    pthread_t threads[nr_of_threads];

    for(int i = 0; i < nr_of_threads; i++)
    {
        pthread_create(&threads[i], NULL, say_hello_thread, (void*) i);
    }

    for(int i = 0; i < nr_of_threads; i++)
    {
        pthread_join(threads[i], NULL);
    }

    return 0;
}
```

```
Hi! I'm thread 0
Hi! I'm thread 3
Hi! I'm thread 2
Hi! I'm thread 5
Hi! I'm thread 1
Hi! I'm thread 4
Hi! I'm thread 6
Hi! I'm thread 7
Hi! I'm thread 8
Hi! I'm thread 9
Hi! I'm thread 10
Hi! I'm thread 11
Hi! I'm thread 13
Hi! I'm thread 12
Hi! I'm thread 14
Hi! I'm thread 16
Hi! I'm thread 17
Hi! I'm thread 18
Hi! I'm thread 19
Hi! I'm thread 15
```

Erzeugt werden die Threads in der richtigen Reihenfolge, also von 0 bis 19 - aber die Reihenfolge der Ausführung wird vom Betriebssystem gemanaged

Threads/IPC

- Interprocess Communication
- Drei Themen
 - Informationsweitergabe
 - Austausch von Informationen zwischen Prozessen
 - Synchronisation
 - Warten auf andere Prozesse
 - Abhängigkeiten/Reihenfolgen
 - Mehrere Prozesse, die in einer definierten Reihenfolge eine Aufgabe abarbeiten sollen (→ Race Conditions)
- Nur der erste Punkt betrifft Threads nicht → selber Adressraum
- Daraus ergeben sich eine Reihe von Problemen und Lösungen: Race Conditions, Critical Regions, Mutex, Semaphoren → LVA VPS

Scheduling

- Bei PCs / mobilen Geräten laufen mehr Prozesse als CPUs zur Verfügung stehen
- Die Prozesse müssen sich daher die Ressource CPU teilen
- Scheduler ist Teil des OS
 - Entscheidet welcher Prozess als nächstes ausgeführt wird
 - Diese Berechnung der Entscheidung wird auch als Scheduling bezeichnet und durch den Scheduling Algorithmus, den der Scheduler implementiert, getroffen
 - Das Resultat der Berechnung führt zu einem Context Switch

Ziele

- Wir werden uns auf interaktive Systeme fokussieren
 - Im Gegensatz zu Batchverarbeitungssystemen oder Echtzeitsystemen
- Ziele
 - Fairness
 - Vergleichbare Prozesse sollten auch den gleichen Anteil an Rechenzeit bekommen
 - Policy Enforcement
 - Unterstützt das OS Policies (z.B. Prioritäten, Deadlines), sollten diese auch eingehalten werden
 - Balance
 - Optimale Ausnutzung der HW Ressourcen
 - Gleichmäßige Auslastung der CPU und der I/O Geräte ist zu bevorzugen
 - Response Time
 - Minimierung der Zeit zwischen einer Benutzereingabe und einer dementsprechenden Rückmeldung
 - Vorrang gegenüber z.B. Hintergrundtasks (Daemons/Services)

Things to consider

- Context Switch ist nicht gratis
 - Wechsel in den Kernel Space, Speichern des PCBs,...
 - Viele Context Switches kosten Performance
 - Steht im Gegensatz zu Ziel Response Time
- Verhalten von Prozessen
 - CPU-bound: Verbringen die meiste Zeit mit Berechnungen
 - I/O-bound: Verbringen die meiste Zeit mit Ein- und Ausgabe
 - Scheduler muss auf das Verhalten von Prozessen Rücksicht nehmen
 - z.B. Bevorzugung von I/O-bound Prozessen, damit diese die Festplatte „beschäftigen“, während CPU-bound Prozesse die CPU beschäftigen → Ziel Balance

Zeitpunkte

- Es gibt mehrere Ereignisse die einen Context Switch auslösen
 - Wenn ein neuer Prozess erzeugt wird
 - Ausführung des Elternprozesses oder des neuen Prozesses
 - Beendigung eines Prozesses
 - Wenn ein Prozess blockiert
 - Blockierendes I/O (System Call), Semaphore, Mutex,...
 - Auftreten eines I/O Interrupts
 - Wurde eine I/O Operation fertig, kann ein Prozess, der auf diese wartet, weiter ausgeführt werden (blockierend → rechenbereit)
 - Oder es könnte trotzdem der aktuelle Prozess weiter ausgeführt werden weil er z.B. eine höhere Priorität hat

Context Switch – Am Beispiel eines Interrupts

- Ein IRQ tritt auf
 - CPU prüft nach der Ausführung der aktuellen Instruktion ob ein IRQ aufgetreten ist
- CPU stoppt Ausführung des aktuellen Prozesses und legt Instruction Pointer etc. auf einen Stack
- CPU lädt mit Hilfe der Interrupt Descriptor Table (IDT) den Interrupt Handler
 - Sichert Register des zuvor ausgeführten Prozesses und die Daten die zuvor von der CPU auf einen Stack gelegt wurden
 - Richtet Stack ein
 - Führt das Interrupt Handling aus (z.B. Lesen von Daten in einen Puffer)
 - Bereinigt Stack/Register
- Aufruf des Schedulers
 - Selektion des nächsten Prozesses
- Ausführen des selektierten Prozesses
 - Laden des Instruction Pointers, Registers,...

Clock Interrupts

- HW Uhr stellt periodische (I/O) Interrupts zur Verfügung
 - Preemptive Scheduler verwenden diese (auch), um über Context Switches zu entscheiden
- Non-preemptive Scheduling (macht man eher nicht mehr)
 - Context Switch nur, wenn ein Prozess blockiert oder die CPU „freiwillig hergibt“
- Preemptive Scheduling
 - Prozesse bekommen eine maximale Ausführungszeit (Zeitscheibe, „Quantum“) zugeordnet
 - Wird diese überschritten, wird der Prozess unterbrochen, auch wenn dieser noch rechenbereit ist, und ein anderer Prozess gestartet
 - Standard bei interaktiven Systemen

Schedulingalgorithmen

- Unterschiedliche OS implementieren unterschiedliche Algorithmen
 - Und Speichern auch unterschiedliche Daten im PCB aufgrund welcher eine Entscheidung zur Prozessselektion getroffen wird
- Hängt von dem Einsatzgebiet und der Zielsetzung ab
- Ein Batchsystem verfolgt andere Ziele als ein Echtzeitsystem oder ein interaktives System
 - Daher ist auch der Schedulingalgorithmus zwischen diesen Systemen anders
 - Ein Batchsystem kann z.B. auch mit non-preemptivem Scheduling auskommen
- Wir sehen uns im Folgenden einige Beispiele von Schedulingalgorithmen an

First Come, First Served (FIFO)

- Verwendung z.B. bei Batchsystemen
 - Non-preemptive
 - Neue Prozesse werden in eine Warteschlange (Queue) eingereiht
 - Prozesse werden der Reihe nach entnommen und abgearbeitet
 - Nachteil: Ziel der Balance wird nicht gewährleistet bei I/O-bound Prozessen

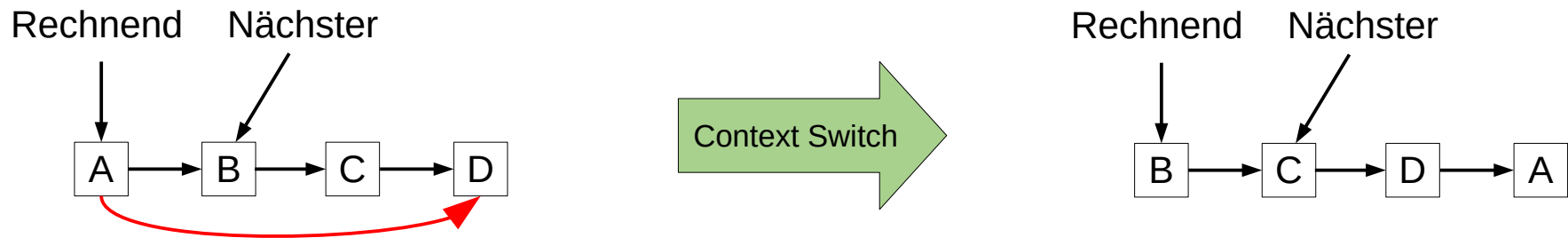
Round Robin

- Jeder Prozess bekommt das gleiche Quantum
 - Läuft er noch nach Ablauf des Quantums, wird er unterbrochen und ein anderer Prozess wird fortgesetzt (preemptive)
 - Blockiert der Prozess früher, wird ebenfalls gewechselt
- Daraus resultiert: Jeder Prozess ist gleich wichtig
 - Vorteil: Fair
 - Nachteil: Vielleicht nicht ganz richtig 
- Weiterer Nachteil – Wahl des Quantums:
 - Kleines Quantum → Verschwendung von CPU Zeit
 - Großes Quantum → Beeinträchtigung der Interaktivität

(weil es cpu bound processes und I/O bound processes gibt
- I/O muss bevorzugt werden, was hier aber nicht gemacht wird)

kann erweitert werden und dann sinnvoll sein. Ist ein basic scheduling algos, den man kennen soll

Round Robin



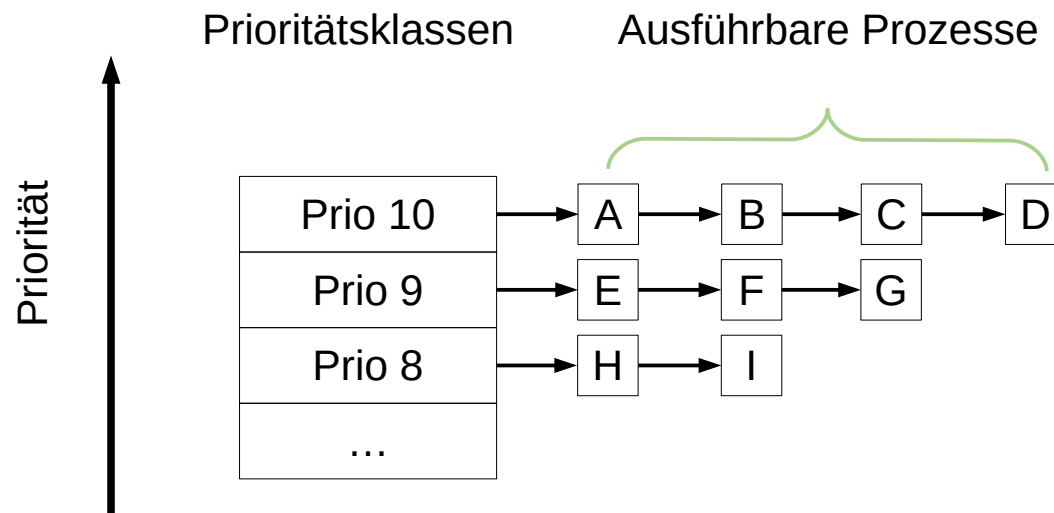
- Scheduler verwaltet eine Liste von Prozessen
- Nach der Ausführung wird der Prozess an das Ende der Liste gereiht und der nächste Prozess in der Liste ausgeführt
- Einfach zu implementieren
 - Wurde von Linux in frühen Versionen (0.x – 1.x) verwendet

Prioritäts-Scheduling

- Prozesse erhalten eine Priorität
 - Höhere Priorität bedeutet, dass diese Prozesse öfter ausgeführt werden und damit auch mehr Rechenzeit bekommen
- Statisch
 - Priorität wird festgelegt und bleibt unverändert
 - z.B. Webbrowser vs. periodischer E-Mail Check
 - z.B. Multiuser Systeme: General vs. Soldat
- Dynamisch
 - Priorität wird vom OS dynamisch angepasst
 - Setzt voraus, dass das OS z.B. verbrauchte Quantumzeit aufzeichnet
 - I/O-bound Prozesse bekommen eine höhere Priorität → Werden schneller/häufiger ausgewählt weil sie die CPU nur kurz belegen → Ziel Balance

Prioritäts-Scheduling - Beispiel

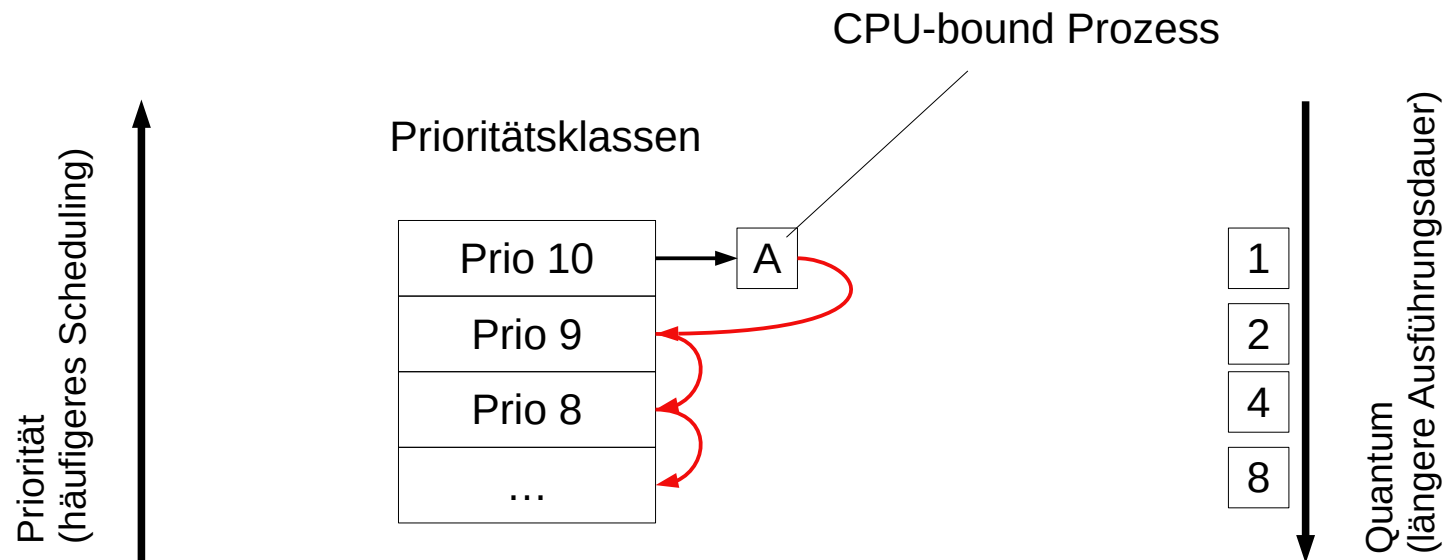
- f : Zeit (anteilig) des letzten Quantums, die für die Berechnung verwendet wurde
 - z.B. Quantum: 100; I/O-bound: $10/100 = 0,1$; CPU-bound: $100/100 = 1$
- $1/f$: Priorität
- CPU-bound: Kleine Priorität, z.B. $1/1 = 1$
- I/O-bound: Hohe Priorität, z.B. $1/0,1 = 10$
- Gruppierung in Prioritätsklassen
 - Round Robin innerhalb dieser Klassen solange die Prozesse der Klasse rechenbereit sind
 - Ignorieren der anderen Klassen, Wechsel in die niederpriore Klasse erst, wenn keine Prozesse mehr rechenbereit sind
 - Nachteil: Verhungern von niederprioren Prozessen → Dynamische Anpassung der Priorität notwendig



Multilevel Feedback Queues (MLFQ)

- Bis jetzt haben wir festgestellt, dass I/O-bound Prozesse eher eine höhere Priorität bekommen sollen
- Erweiterung nun bei CPU-bound Prozessen
 - Besser diese für längere Zeit rechnen zu lassen, als oft für kurze Zeit
 - Minimierung der Context Switches
- Daher: Prozessen in unterschiedlichen Prioritätsklassen wird eine unterschiedliche Anzahl an Quanten zugeteilt
 - Höchste Prio Klasse: 1 Quantum
 - Nächste Prio Klasse: 2 Quanten
 - Nächste Prio Klasse: 4 Quanten
 - ...
- Je nachdem wie viel ein Prozess rechnet oder auf I/O wartet, wird dieser zwischen den Klassen verschoben und
 - damit die Priorität geändert
 - die Menge an CPU Zeit, die der Prozess erhält, dynamisch angepasst

MLFQ - Beispiel



- Beispiel: Prozess A benötigt 20 Quanten und kein I/O
 - Startet in Prioritätsklasse 10 und wird unterbrochen → kann nicht fertig rechnen
 - A wird daher nach unten gereiht (CPU-bound)
 - A wird das nächste Mal in Prioritätsklasse 9 gestartet
 - A bekommt daher weniger häufig die CPU
 - A bekommt dafür die CPU für eine längere Zeit
- Bsp: Was würde passieren wenn A plötzlich beginnt I/O Operationen durchzuführen?
- Bsp: Vergleich Round Robin: 5 vs. 20 Context Switches

Implementierungen

- OS verwenden oft Varianten/Mischformen von MLFQ
 - Mac OS X, (Net|Free)BSD + Prioritäten
- Windows
 - 3.1x: Non-preemptive Scheduler
 - 95: Preemptive Scheduler
 - >= Vista: MLFQ + Magic (Closed Source)
 - Windows Scheduler arbeitet mit Threads
 - Threads besitzen zusätzlich eine statische Priorität (kann von BenutzerIn oder im Code gesetzt werden)
 - Priorität kann aber auch vom OS angepasst werden
 - Ausnahme: Threads der Real Time Prioritätsklasse

Implementierungen - Linux

- Linux 0.x – 1.x: Round Robin
- Linux 2.2: Erweiterungen Scheduling Klassen und SMP Support
- Linux 2.4: $O(n)$ Scheduler
 - Iteration über alle Prozesse der Run Queue $\rightarrow O(n)$
 - Auswahl des nächsten Prozesses aufgrund einer Heuristik
 - Mitnahme von nicht verbrauchtem Quantum (wenn Prozess blockiert)
- Linux 2.6: $O(1)$ Scheduler
 - Auswahl des nächsten Prozesse aus der Run Queue in $O(1)$
 - Allerdings viele Heuristiken zur Feststellung ob ein Prozess CPU-bound oder I/O-bound ist $\rightarrow$ komplex
- Linux 2.6.23: Completely Fair Scheduler (CFS)
 - Verhalten kann durch Scheduling Policies verändert werden (neben dem standard, interaktiven Scheduling gibt es auch noch FIFO, Round Robin und Deadline)
 - Deadline Policy ist für Echtzeitsysteme ausgelegt

Implementierungen - Linux

- Linux 6.6: EEVDF Scheduler
 - Earliest Eligible Virtual Deadline First¹
 - In CFS wurden mit der Zeit auch wieder Heuristiken und Parameter eingebaut um kleinere Schwächen auszumerzen
 - Hauptproblem: Latenz (Interaktivität)
 - Manche Prozesse benötigen wenig Rechenzeit
 - Aber wenn sie diese benötigen, dann sollten sie sie schnell bekommen
 - In CFS keine Möglichkeit dies auszudrücken/einzustellen
- Linux 6.12
 - Echtzeit Unterstützung (PREEMPT_RT) wurde nach 20 Jahren Entwicklung fertiggestellt
 - Finalisierung EEVDF
 - sched_ext: Möglichkeit, Scheduler als User-Space Programme zu implementieren
- Alternative: Brain Fuck Scheduler (BFS) / MuQSS (Con Kolivas), SCX, LAVD,...

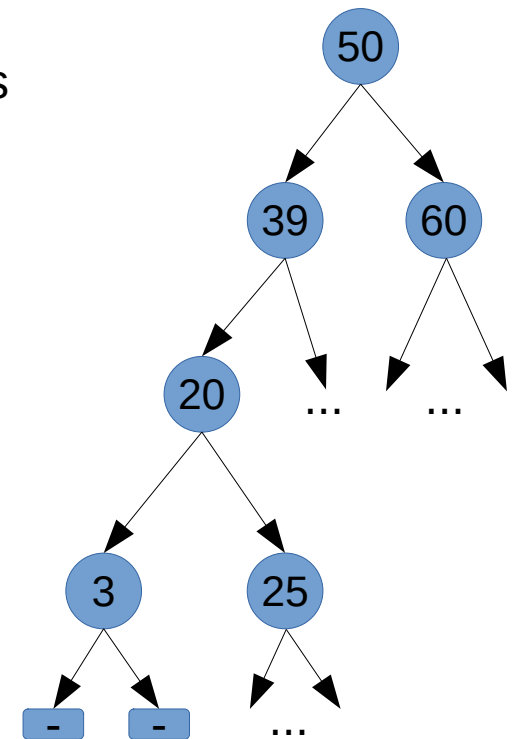
¹<https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=805acf7726282721504c8f00575d91ebfd750564>

Completely Fair Scheduler

- *CFS basically models an 'ideal, precise multitasking CPU' on real hardware. CFS's design is quite radical: it does not use runqueues, it uses a time-ordered rbtree to build a 'timeline' of future task execution* -Ingo Molnár
- „Fair“ weil jeder Prozess Anspruch auf gleich viel Prozessorzeit hat
 - Jeder Prozess bekommt also den selben Anteil an CPU Zeit unter Berücksichtigung der anderen Prozesse (Maximum Execution Time)
 - Maximum Execution Time für einen Prozess wächst mit der Dauer, die er wartend verbringt („Sleeper Fairness“)
 - Die Zeit vergeht allerdings für low-prio Prozesse schneller
 - Da sich die Anzahl an Prozessen laufend ändert, gibt es kein fixes Quantum für einen Prozess
- Für jeden Prozess wird die Zeit, die er ausgeführt wird, aufgezeichnet (`vruntime`)
- Es gibt keine spezielle Behandlung oder Heuristik zur Feststellung der Interaktivität eines Prozess (→ einfacher als $O(1)$ Scheduler)

Completely Fair Scheduler

- Die Datenstruktur (Run Queue) für die Prozesse ist ein Rot-Schwarz Baum, der nach der `vruntime` sortiert ist
 - Knoten mit der kleinsten `vruntime` befinden sich im Baum links unten
- Es wird immer der linkeste Knoten (Prozess) entnommen und als nächstes ausgeführt
 - Der Prozess wird für Maximum Execution Time ausgeführt
 - Die `vruntime` des Prozesses wird aktualisiert und der Prozess nach der Ausführung wieder im Baum nach der `vruntime` sortiert eingefügt
 - Während ein Prozess ausgeführt wird, wächst die Maximum Execution Time der anderen Prozesse
 - Damit gibt es einen neuen linken Knoten welcher als nächstes entnommen und ausgeführt wird
- CFS wurde über die Jahre weiterentwickelt
 - Auto-group von Prozessen, die logisch zusammengehören, um Fairness zu steigern
 - Verbesserungen im Bereich Multicore Performance



EEVDF

EEVDF Algorithm. *A new quantum is allocated to the client that has the eligible request with the earliest virtual deadline¹*

- Ähnlichkeiten zu CFS
 - Ebenfalls Verwendung virtueller Zeit (flexibles Quantum) -> faire Aufteilung der CPU auf Prozesse unter Berücksichtigung ihrer Priorität
 - Prozesse mit hoher Priorität (= hohe Latenz) bekommen ein kürzeres Quantum
- Aufzeichnen wieviel Zeit wirklich verbraucht wurde: „Lag“
 - Positiv: Nicht alles verbraucht, sollte früher gescheduled werden
 - Prozess hat seinen „fairen“ Anteil nicht bekommen
 - Negativ: Alles oder mehr verbraucht, sollte später gescheduled werden
- „Eligible“ Prozesse sind jene mit einem positiven Lag (≥ 0)
 - Negative Lags werden über die Zeit „abgebaut“
- Virtual deadline: Zeit die dem Prozess zusteht (Quantum) + eligible time (Lag)
- Prozesse mit der frühesten, virtuellen Deadline werden als erstes ausgeführt
 - Damit werden Prozesse mit hoher Latenz öfter ausgeführt

¹<https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=805acf7726282721504c8f00575d91ebfd750564>

EEVDF

▪ Beispiel

- Prozess mit hoher Latenz
 - Quantum: 10
 - Lag: 2
 - -> Virtuelle Deadline: 12
- Prozess mit niedriger Latenz
 - Quantum: 50
 - Lag: 5
 - -> Virtuelle Deadline: 55

▪ Zusätzlich

- Kann ein aufwachender Prozess mit einem kurzen Quantum einen gerade laufenden Prozess mit einem langen Quantum unterbrechen
- Und Prozesse können, wenn Sie möchten, ihr Quantum mit einem System Call selbst bestimmen
 - Setzen eines kürzeren Quantums -> Prozess wird öfter ausgeführt

A Decade of Wasted Cores¹

- Eine Run Queue pro Prozessor/Core
 - Bei einer einzigen globalen Run Queue wäre Locking notwendig
→ Performance
- CPU Affinity für Prozesse
 - Prozesse werden einer CPU/Core zugeordnet
 - Um Cache Misses zu reduzieren
 - Das Verschieben eines Prozesses von einer Run Queue in eine andere ist eine teure Operation
- Load Balancing zwischen CPUs/Cores
 - Ziel ist, alle CPUs/Cores möglichst gleichmäßig auszunutzen
 - Daher wird periodisch ein Load Balancing Algorithmus ausgeführt, der eine Umverteilung der Prozesse, falls benötigt, durchführt

¹<https://people.ece.ubc.ca/sasha/papers/eurosys16-final29.pdf>

Zusammenfassung

- Prozesse
- Threads
- Prozesszustände
- Prozessverwaltung
- Scheduling
- Schedulingalgorithmen